Experiment 13 — Molecular dynamics: the row that needed no fix, and the one line that broke it

Mathilda — execution-speed experiments

Contents

Experiment 13 — Molecular dynamics: the row that needed no fix, and the one line that broke it

Date: 2026-07-31 · Code: no change required · Result: Lennard-Jones 6.9× faster than Mathematica out of the box — and a return statement that cost 120×, one dtype away from the third sweep's defect

Common method in `README.md`.

Hypothesis

The third sweep's N-body row is the closest existing benchmark to molecular dynamics, and it is a different kernel. Gravity has no cut-off, so an all-pairs gravity step is pure arithmetic over a dense matrix. Lennard-Jones multiplies through a data-dependent mask:

```
mk = UnitStep[rc2 - r2];
ff = (48. i6 i6 - 24. i6) i2 mk;
```

The question was whether a comparison result can stay on the buffer, or whether the mask forces the whole interaction matrix back through the evaluator — the failure mode that made `Pick`, `Select` and `Position` slow in the fourth sweep's probe.

The configuration is a deterministically perturbed simple-cubic lattice, so the whole benchmark reproduces across systems rather than only its check, and the check is the potential energy, which every pair contributes to.

The kernels

id	kernel	what it stresses
`ljforce`	LJ force, 2048 atoms, cut-off	masked all-pairs, 4.2 × 10⁶ interactions

id	kernel	what it stresses
`mdverlet`	velocity-Verlet, 10 steps	the above, iterated, with loop-carried state
`mdcell`	cell-list binning, 100000 atoms	`Floor`, `Sort`, `Tally`, `Accumulate`

Results — nothing needed fixing

Benchmark	Mathilda	Mathematica 14.0	NumPy 2.4.4	vs WL	vs NumPy
LJ force, 2048 atoms, cut-off	394 ms	2.732 s	152 ms	6.93×	1/2.59×
Velocity-Verlet, 10 steps	6.50 s	32.23 s	1.73 s	4.96×	1/3.75×
Cell-list binning, 100000 atoms	4.7 ms	1.1 ms	1.4 ms	1/4.20×	1/3.48×

This is the only group in the sweep that required no code change, and it is worth saying why plainly: the mask stays on the buffer. `UnitStep` has a narrowing kernel (real in, exact `int64` out — experiment 5), `Times` is exact on an `int64` buffer, and `0uter` and `Total[…, {2}]` were put on the buffer by the third sweep. Every step of a cut-off interaction is already a machine loop, and Mathilda is ahead of Mathematica by 5–7× on the two rows that matter.

The 2.6–3.8× against NumPy is the unfused-composition gap that experiment 11 named (plan item 9.2): the force expression is nine passes over a 33 MB matrix where a fused kernel would make three.

`mdcell` is the one row behind both, and it is small enough (4.7 ms) that its `Sort+Tally` pair is dominated by the two hash/compare passes rather than by anything structural.

The finding: a return statement, one dtype away

The kernel was first written with a shared helper, because force and energy need the same six intermediates:

```
ljpair[xs_, ys_, zs_] := Module[{...}, {dx, dy, dz, i2, i6, mk}]
```

Five float64 matrices and one `int64` mask. That run had to be killed: it was heading for tens of minutes and hundreds of megabytes.

The third sweep established that a function returning several arrays used to destroy all of them, and fixed it by having `evaluate_step` offer the whole list node for packing — n packed rows of the same shape and class are a rank-(k+1) buffer. What it cannot absorb is rows of different dtype, because widening the `int64` mask to float64 would turn its exact `Integers` into `Reals`. So the list declines, and the no-nesting invariant then requires every element to be materialised.

Measured directly, on 600 × 600 matrices — `molecular_dynamics.m` runs this section, so the numbers below are reproducible rather than quoted:

	cost
`{a, b}` — two float64 matrices	0.55 ms (packed, rank 3)
`{a, b, m}` — plus an `int64` mask	53.0 ms (not packed)
`{a, b, 1. UnitStep[…]}` — mask widened by hand	1.07 ms
the caller's next operation, `q[[1]] + q[[2]]`, uniform	0.90 ms
the same, after the mixed return	107.8 ms

So the cost is 96× at the **`return`** and 120× on the next thing the caller does, and the whole of it turns on one of six values having a different element type. (An earlier single-shot measurement put these at 38× and 89×; the figures above are the minimum of three runs, which is what the rest of the suite reports and what the script prints.) Profiling the slow call points at `0uter` and the arithmetic; all of them are innocent, exactly as in the third sweep.

This is recorded rather than fixed, because the fix is not local. Making `List` packed-aware would let a plain `List` hold EXPR_NDARRAY elements, which is the malformed shape the transparency gate exists to prevent and would turn every unaware head's O(argc) top-level scan into an O(tree) one. Widening the integer row is a value change. The honest options are a real design change (a heterogeneous packed tuple) or a diagnostic that tells the user what happened. The workaround is one character: make the mask `1. UnitStep[…]`.

The benchmark was rewritten so that `ljf` and `lje` are each self-contained and return uniform tuples — which is what an MD code does anyway — so the row measures Lennard-Jones rather than measuring this.

A measurement caveat, stated because it looks like a regression

`mdverlet` reads 5.14 s in the before table and 6.50 s in the after table. That is not a regression. Measured standalone, back to back, on the same host:

	baseline binary	fixed binary
`ljf`, three runs	425, 403, 413 ms	407, 395, 412 ms
Verlet, 10 steps, two runs	4.606, 4.674 s	4.526, 4.570 s

The fixed binary is marginally ahead on every one. Both harness numbers are above both standalone numbers, which is the signature of the allocator and thread state left by the rows that ran before it in the same session — the same class of effect as the `dgemm` interference the fourth sweep documented. This row allocates roughly 300 MB per force evaluation, so it is the most sensitive in the suite to that; its run-to-run spread is about ±20%. The `kalman` and `enkf` rows in experiment 18 carry the same caveat.

Why Mathilda is not the fastest here, and what it would take

Mathilda leads Mathematica by 6.9× and 5.0× on the two rows that matter, and trails NumPy by 2.6–3.8×.

row	Mathilda	best other	gap	cause
Lennard-Jones force	394 ms	152 ms (NumPy)	2.59×	nine unfused passes over a 33 MB matrix
Velocity-Verlet, 10 steps	6.50 s	1.73 s (NumPy)	3.75×	the above, ten times
Cell-list binning	4.7 ms	1.1 ms (WL)	4.20×	`Sort` then `Tally` — two hash/compare passes

The road to fastest

1. Fuse the interaction expression (plan 9.2). The force is `r2`, `mk`, `i2`, `i6`, `i6 i6`, the bracket, two products and three reductions — nine passes over 4.2×10^6 float64, 33 MB each, where a fused kernel makes three. `Compile[]` already fuses exactly this shape (experiment 2); ordinary array code cannot reach it. This is the whole of the 2.6× and it is the same item as experiments 14, 16 and 19.
2. A fused masked reduction. `Total[ff dx, {2}]` allocates a full 4.2×10^6 product to sum it away immediately, three times over. A `Total[a b, {2}]` recognition — a row-wise dot product — removes three 33 MB temporaries from every force evaluation.
3. **`Tally`** without the **`Sort`**. The binning row sorts and then tallies; `ndred_tally` already hashes machine words, so the `Sort` is redundant whenever the caller only wants counts. 4.7 ms → ~1.5 ms.
4. A packed heterogeneous tuple — see experiment 17's roadmap. It does not affect the rows above as written, but it is what makes the natural spelling of this kernel (one helper returning the six shared intermediates) cost 3 ms instead of 112 ms. Until it exists, the workaround stands: keep every returned tuple uniform, widening an integer mask with `1. UnitStep[…]` if need be.

With 1 and 2 the LJ row should reach ~150 ms, i.e. level with NumPy, and the Verlet row follows it. Nothing here is a missing fast path — every operation is already on the buffer — so this experiment's roadmap is entirely about how many times the data is touched.

Verification

- The energy check is deterministic and agrees to full printed precision across all three systems, before and after the kernel was restructured.
- No source change was made for this experiment; the differential tests that cover the paths it exercises are the third and fourth sweeps'.